

Documentation Technique

SAE S5



BAJON CONSULTING

IT Strategy for your Business

Nom : Clavel, Kwadjani, Cornu, Gourgues, Gauze

Informatique 3^{ème} Année

Sommaire

Contextualisation.....	4
1. Objectif principal de l'application.....	4
1.1. Objectifs fonctionnels de la V1.....	5
1.2. Objectifs techniques de la V1.....	5
1.3. Authentification et accès.....	6
2. Architecture du projet.....	7
2.1. Frontend.....	7
2.1.1. Racine du projet.....	7
2.1.2. src/ – Code source principal.....	8
2.1.3. assets/.....	8
2.1.4. components/.....	9
2.1.5. feature/ – Cœur métier (architecture par domaine).....	10
a. audits.....	10
b. auth.....	11
c. clients.....	11
d. templates.....	12
e. users.....	12
2.1.6. layouts/.....	12
2.1.7. routes/.....	13
2.1.8. lib/.....	13
2.1.9. types/.....	13
1.2. Backend.....	14
1.2.1 Bajan_audit_App.....	14
1.2.2 config/.....	15
1.2.3 controller/.....	15
1.2.4 generator/.....	16
1.2.5 service.....	16
1.2.6 type.....	16
1.2.7 ressource/.....	18
1.3. Base de donnée.....	19
1.3.1 Choix technologique.....	19
1.3.2 Modèle de données général.....	19
1.3.3 Schéma relationnel de la base de données.....	20
1.3.4 Description des tables principales.....	20
Table client.....	20
Table Audit.....	21
Utilisateurs (Supabase Auth).....	21
Tables Theme, Question, OptionAnswer.....	21

Table ClientAnswer.....	22
Tables de liaison (relations).....	22
1.3.5 Sécurité et intégrité des données.....	22
3. Use case.....	23
2.1 [US] Connexion Utilisateurs.....	23
2.2 [US] CRUD Client.....	23
2.3 [US] Tableau de bord administrateur.....	24
2.4 [US] CRUD Utilisateur.....	25
2.5 [US] Validation des audits traités.....	26
2.6 [US] Création et gestion d'audits.....	27
2.7 [US] Création et gestion des templates d'audit.....	28
2.8 [US] Création d'audit à partir d'un template.....	28
2.9 [US] Consultation des résultats d'audit.....	29
2.10 [US] Consultation des audits.....	29
4. Description des fonctionnalités.....	30
3.1. Audit.....	30
3.1.1. Créer.....	30
3.1.2. Modifier.....	30
3.1.3. Consulter.....	30
3.1.4. Archiver.....	30
3.2. Template.....	30
3.2.1. Créer.....	30
3.2.2. Modifier.....	31
3.2.3. Consulter.....	31
3.3. Clients.....	31
3.3.1. Créer.....	31
3.3.2. Modifier.....	31
3.3.3. Supprimer.....	31
3.4. Users.....	31
3.4.1. Créer.....	31
3.4.2. Modifier.....	31
3.4.3. Supprimer.....	32
5. Déploiement et exécution de l'application.....	32
5.1. Principe général.....	32
5.2. Prérequis techniques.....	32
5.3. Architecture de déploiement.....	32
5.4. Accès à l'application.....	33
5.5. Configuration de l'environnement.....	33
6. Sécurité et gestion des accès.....	34
6.1. Authentification.....	34
6.2. Rôles et permissions.....	34

Administrateur.....	34
6.3. Protection des accès.....	35
Limite et évolution.....	35
Synthèse.....	36

Contextualisation

Ce projet consiste en la conception et le développement d'une application web interne destinée au client Bajon Consulting, une société spécialisée dans l'audit de sécurité et de maturité numérique en informatique.

L'application vise à répondre aux besoins internes de l'entreprise en matière de gestion, suivi et restitution des audits clients, dans un contexte de transformation numérique et de professionnalisation des processus d'audit.

Elle est conçue pour être sécurisée, évolutive et ergonomique, tout en garantissant la confidentialité des données sensibles manipulées lors des missions d'audit.

L'application est strictement réservée à un usage interne :
les clients finaux de Bajon Consulting n'ont pas accès à la plateforme.
Seuls les collaborateurs de l'entreprise (auditeurs et administrateurs) utilisent l'outil pour préparer, réaliser et analyser les audits.

Le contact client référent pour le projet est :
Christophe Bajon, dirigeant de Bajon Consulting.

1. Objectif principal de l'application

L'objectif principal du projet est de concevoir et développer une application web responsive (PC, tablette, mobile) permettant :

- la gestion centralisée des audits clients
- la structuration et la standardisation des formulaires d'audit
- le suivi de l'état d'avancement des audits
- la préparation des livrables de restitution

La première offre ciblée par l'application est l'offre "**Essentiel**", qui constitue la base des audits proposés par Bajon Consulting. L'architecture retenue permet toutefois d'envisager facilement l'ajout d'offres plus avancées dans les versions futures.

1.1. Objectifs fonctionnels de la V1

La version V1 de l'application a pour objectif de mettre en place **les fonctionnalités fondamentales**, nécessaires au fonctionnement minimal et cohérent de la plateforme.

Les fonctionnalités principales implémentées en V1 sont :

- **Gestion des clients**

Création, consultation, modification et suppression des clients (CRUD Client)

- **Gestion des utilisateurs**

Création, modification et suppression des comptes utilisateurs (CRUD Utilisateur)

- **Gestion des audits**

Création, consultation, modification et archivage des audits (CRUD Audit)

- **Gestion des templates d'audit**

Création, consultation et modification des modèles d'audit (CRUD Template)

Formulaires d'audit

Création de formulaires structurés par thèmes et questions

Saisie et stockage des réponses associées à un audit

Cette V1 constitue une **base fonctionnelle solide**, sur laquelle pourront s'appuyer les évolutions futures (analyses avancées, graphiques détaillés, export PDF, recommandations automatiques, etc.).

1.2. Objectifs techniques de la V1

Sur le plan technique, la V1 vise à mettre en place une **architecture modulaire et maintenable** et séparer clairement les responsabilités entre :

- frontend (React + TypeScript)
- backend (API REST Spring Boot)
- base de données (Supabase / PostgreSQL)

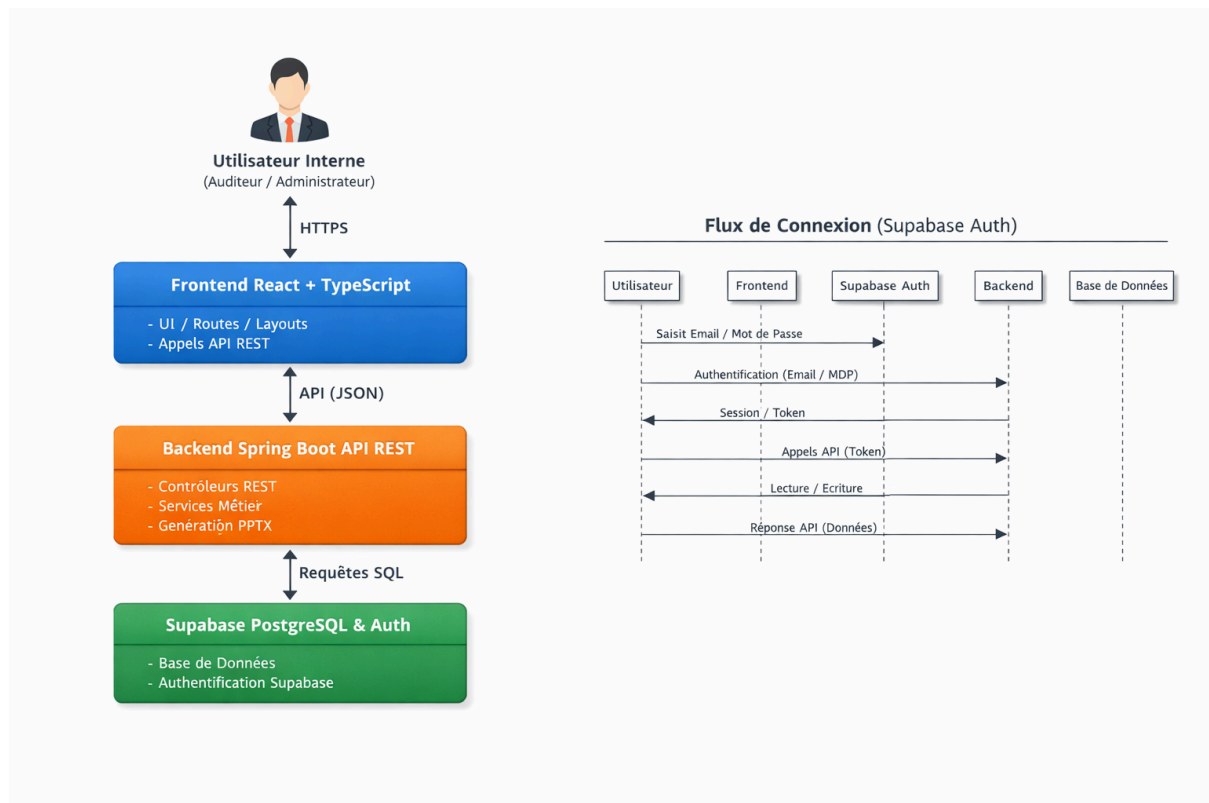
Elle doit aussi assurer un **haut niveau de sécurité**, notamment via l'authentification fournie par Supabase qui garantit la protection des accès aux données et de garantir une **application responsive**, utilisable sur ordinateur, tablette et mobile ainsi que de préparer l'application à des **évolutions futures sans refonte majeure**.

1.3. Authentification et accès

L'authentification des utilisateurs repose sur le système d'authentification fourni par Supabase, utilisé via son API dédiée.

Dans la V1 le mode d'authentification implémenté est l'authentification par identifiant (adresse e-mail) et mot de passe et la gestion des sessions et de la sécurité est déléguée à Supabase et seuls les utilisateurs authentifiés peuvent accéder à l'application

Les mécanismes d'authentification avancés (SSO, OAuth, multi-facteurs) ne sont pas intégrés dans la V1 mais pourront être envisagés dans des versions ultérieures.



2. Architecture du projet

2.1. Frontend

Ce frontend est une application React + TypeScript, organisée selon une architecture par features (domain-driven), avec séparation claire entre :

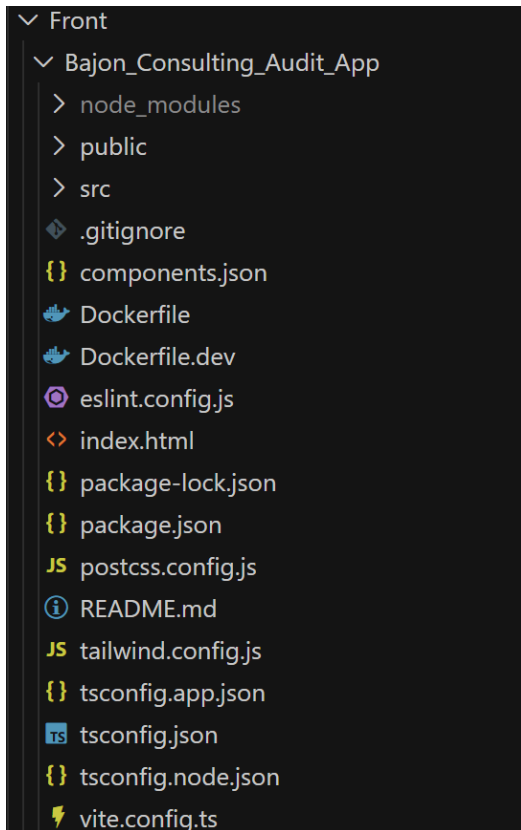
- composants génériques réutilisables
- fonctionnalités métier (audits, users, clients...)
- routes et layouts
- services transverses (auth, supabase, types)

Cette organisation favorise :

- la lisibilité
- la maintenabilité
- la scalabilité (ajout de nouvelles features sans dette technique)

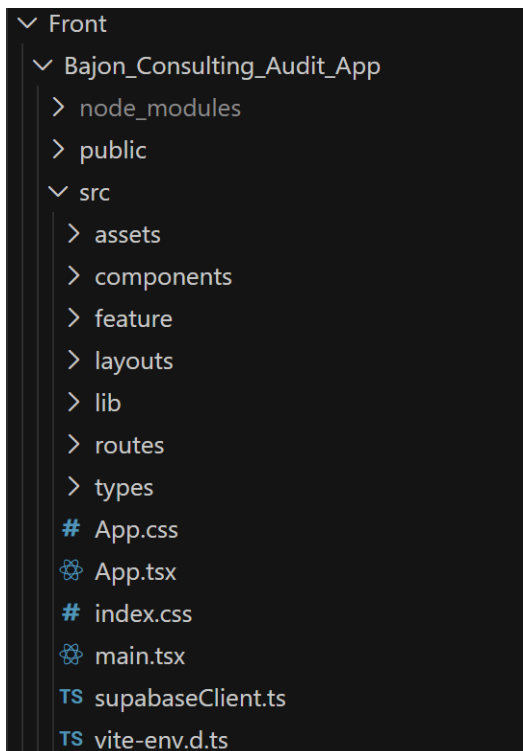
2.1.1. Racine du projet

Cette arborescence présente la racine du projet frontend. Elle regroupe les fichiers de configuration globaux de l'application (gestion des dépendances, configuration TypeScript, outils de build) ainsi que le dossier src/ qui contient l'ensemble du code source de l'application React.



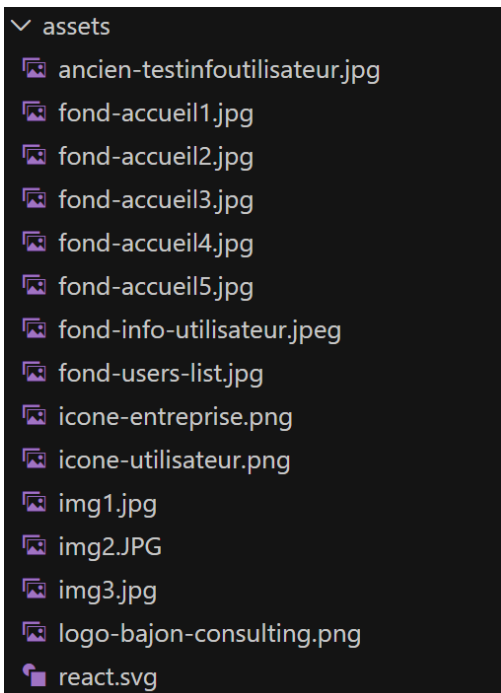
2.1.2. src/ – Code source principal

Le dossier src/ contient tout le code source de l'application frontend. Il regroupe les composants React, la logique métier, la gestion des routes, les layouts, les ressources et les services transverses. C'est le cœur du frontend.



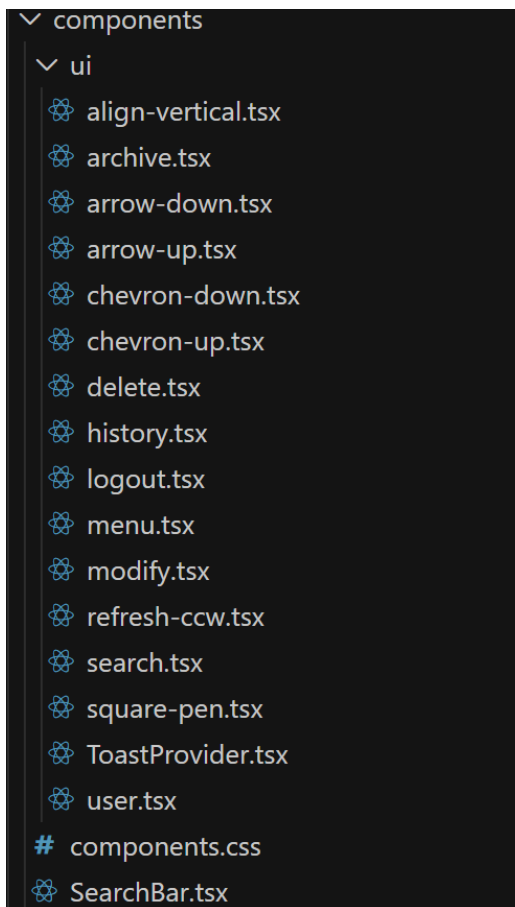
2.1.3. assets/

Le dossier assets/ contient les ressources statiques utilisées par l'application, telles que les images, icônes ou autres fichiers graphiques. Centraliser ces éléments permet de faciliter leur gestion et leur réutilisation dans les composants.



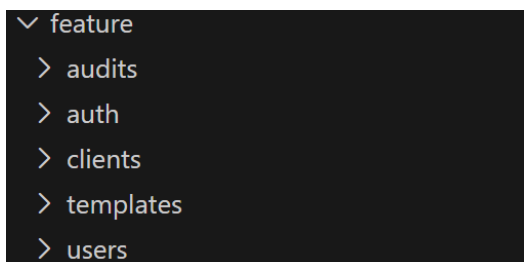
2.1.4. components/

Ce dossier regroupe les composants génériques et réutilisables de l'application. Ces composants ne sont pas liés à une fonctionnalité métier spécifique et peuvent être utilisés dans plusieurs parties du projet (boutons, modales, tableaux, champs de formulaire, etc.).



2.1.5. feature/ – Cœur métier (architecture par domaine)

Le dossier feature/ contient la logique métier principale de l'application, organisée par domaine fonctionnel. Chaque sous-dossier correspond à une fonctionnalité clé du projet et regroupe ses composants, services et logiques associées, selon une approche feature-based.



a. audits

Ce dossier contient l'ensemble des fichiers liés à la gestion des audits : affichage de la liste, création, modification, consultation et archivage des audits.

```
✓ audits
  ✓ AuditCard
    # AuditCard.css
    AuditCard.tsx
  ✓ AuditForm
    AuditForm.tsx
    # AuditStyle.css
    questionBox.tsx
    sectionBox.tsx
    sideBarNav.tsx
  ✓ AuditListPage
    # AuditListPage.css
    AuditListPage.tsx
  ✓ NewAudit
    # NewAudit.css
    NewAudit.tsx
  AuditEditorPage.tsx
  TS auditService.ts
```

b. auth

Le dossier auth regroupe tout ce qui concerne l'authentification des utilisateurs, notamment la gestion de la connexion, des sessions et des différents moyens d'accès à l'application.

```
✓ auth
  # auth.css
  TS AuthContext.ts
  AuthProvider.tsx
  LoginPage.tsx
  TS useAuth.ts
```

c. clients

Ce dossier contient les fonctionnalités liées à la gestion des clients, incluant la création, la modification et la suppression des informations clients.

```
▼ clients
  ⚙ ClientCard.tsx
  ⚙ ClientEdit.tsx
  ⚙ ClientInfo.tsx
  ⚙ ClientListPage.tsx
  # clients.css
```

d. templates

Le dossier templates regroupe la logique liée aux modèles d'audit. Il permet aux administrateurs de créer, modifier et consulter des templates servant de base à la création des audits.

```
▼ templates
  ▼ TemplateListPage
    # TemplateListPage.css
    ⚙ TemplateListPage.tsx
    ⚙ TemplateEditorPage.tsx
```

e. users

Ce dossier est dédié à la gestion des utilisateurs de l'application. Il inclut les fonctionnalités de création, modification, suppression et consultation des comptes utilisateurs.

```
▼ users
  ⚙ UserCard.tsx
  ⚙ UserForm.tsx
  ⚙ UserInfo.tsx
  ⚙ UserListPage.tsx
  # users.css
```

2.1.6. layouts/

Le dossier layouts/ contient les structures globales d'affichage de l'application. Les layouts définissent l'organisation générale des pages (header, sidebar, contenu principal) et sont partagés entre plusieurs routes.

```
▼ layouts
  ⚙ BaseLayout.tsx
  ⚙ DashboardLayout.tsx
  ⚙ Footer.tsx
  ⚙ header.tsx
  # layouts.css
```

2.1.7. routes/

Le dossier routes/ centralise la définition des routes de l'application. Il permet de gérer la navigation entre les différentes pages et de lier chaque route à son composant correspondant.

```
▼ routes
  ⚙ AppRoutes.tsx
```

2.1.8. lib/

Le dossier lib/ regroupe les services et outils transverses utilisés dans plusieurs parties du projet, comme la configuration de Supabase, l'authentification ou les appels aux services externes.

```
▼ lib
  TS utils.ts
```

2.1.9. types/

Le dossier types/ contient les définitions TypeScript partagées dans toute l'application. Il permet d'assurer une cohérence des types et de renforcer la sécurité et la lisibilité du code.

```
▼ types
  TS audit.d.ts
  TS editionMode.ts
```

1.2. Backend

Le backend de l'application est une API REST développée en Java avec le framework Spring Boot.

Il assure la gestion de la logique métier, la sécurité des accès, la communication avec la base de données ainsi que la génération de documents de restitution (PowerPoint).

L'architecture du backend repose sur une séparation claire des responsabilités, facilitant la maintenabilité, l'évolutivité et la lisibilité du code. Les différentes couches de l'application sont organisées en packages distincts, chacun ayant un rôle bien défini.

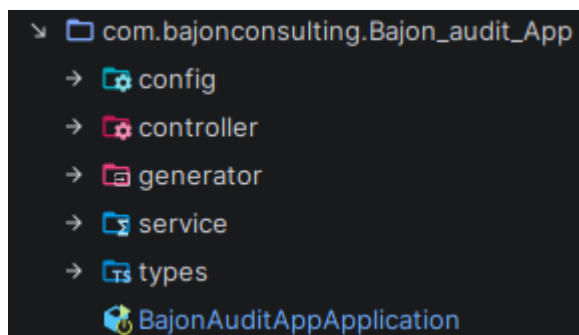
Cette organisation permet notamment :

- une meilleure structuration du code
- une séparation entre logique métier et exposition de l'API
- une évolution simplifiée des fonctionnalités backend

1.2.1 Bajon_audit_App

Le package `Bajon_audit_App` constitue le package principal de l'application backend. Il contient l'ensemble des sous-packages du projet ainsi que la classe principale permettant le lancement de l'application Spring Boot.

Ce package représente le point d'entrée global du backend.



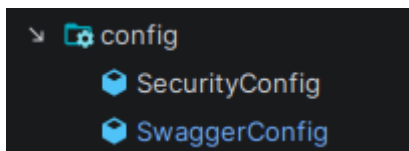
1.2.2 config/

Le package config/ regroupe toutes les **classes de configuration** de l'application.

Il contient notamment :

- la configuration de la sécurité (SecurityConfig), définissant les règles d'accès l'API backend
- la configuration Swagger (SwaggerConfig), permettant de générer automatiquement une documentation interactive de l'API, consultable via une URL dédiée lorsque l'application est lancée

Ces configurations permettent de centraliser les paramètres techniques et transverses du backend.



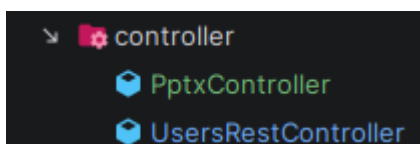
1.2.3 controller/

Le package controller/ contient l'ensemble des contrôleurs REST, qui définissent les points d'entrée de l'API.

Les contrôleurs reçoivent les requêtes HTTP, valident les données entrantes et délèguent le traitement à la couche service.

Dans le cadre de ce projet, les contrôleurs exposent notamment

- une API de gestion des utilisateurs (CRUD)
- une API permettant la génération et le téléchargement des fichiers PowerPoint (PPTX)

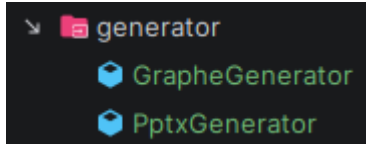


1.2.4 generator/

Le package generator/ regroupe les classes responsables de la génération de documents et de contenus dynamiques.

Dans notre application, ce package est utilisé pour générer des fichiers PowerPoint contenant des graphiques, destinés à la restitution des audits auprès des clients.

Cette logique est isolée afin de faciliter son évolution ou son remplacement ultérieur.

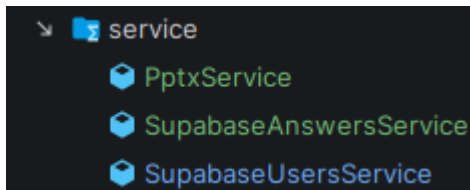


1.2.5 service

Le package service/ contient l'ensemble des classes de logique métier de l'application.

Les services implémentent les règles fonctionnelles et orchestrent les traitements nécessaires en s'appuyant sur les données et les composants techniques.

Ils sont appelés par les contrôleurs et constituent le cœur fonctionnel du backend.



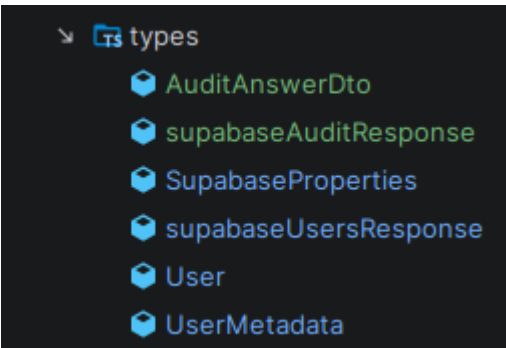
1.2.6 type

Le package type/ contient les classes représentant les entités métier et les objets échangés avec la base de données.

Le package types/ regroupe les classes représentant les types d'objets manipulés par l'application, notamment

- les entités métier liées à la base de données
- les objets de transfert de données (DTO) utilisés pour les échanges entre le backend et le frontend

Cette séparation permet de garantir une meilleure cohérence des données et de sécuriser les informations exposées par l'API



```
types
├── AuditAnswerDto
├── supabaseAuditResponse
├── SupabaseProperties
├── supabaseUsersResponse
├── User
└── UserMetadata
```

1.2.7 ressource/

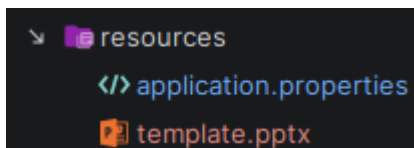
Le dossier resources/ contient les ressources non compilées utilisées par l'application backend.

Il regroupe les fichiers de configuration et les ressources statiques nécessaires au bon fonctionnement du backend.

On y trouve notamment :

- le fichier application.properties, qui centralise la configuration de l'application (connexion à la base de données, paramètres de sécurité, ports, variables d'environnement, etc.)
- le fichier template.pptx, qui sert de modèle de base pour la génération des fichiers PowerPoint utilisés lors de la restitution des audits

Le fait de placer ces fichiers dans le dossier resources/ permet à Spring Boot de les charger automatiquement au démarrage de l'application et de les rendre accessibles aux différentes couches du backend.



1.3. Base de donnée

1.3.1 Choix technologique

La base de données de l'application est sur PostgreSQL, via la plateforme Supabase.

Ce choix permet de bénéficier d'une base relationnelle robuste, sécurisée et adaptée à des données structurées et sensibles.

Supabase fournit également un système d'authentification intégré, utilisé pour la gestion des utilisateurs, ainsi que des mécanismes de sécurité avancés (Row Level Security) permettant de contrôler précisément l'accès aux données.

1.3.2 Modèle de données général

Le modèle de données est organisé autour des entités principales suivantes :

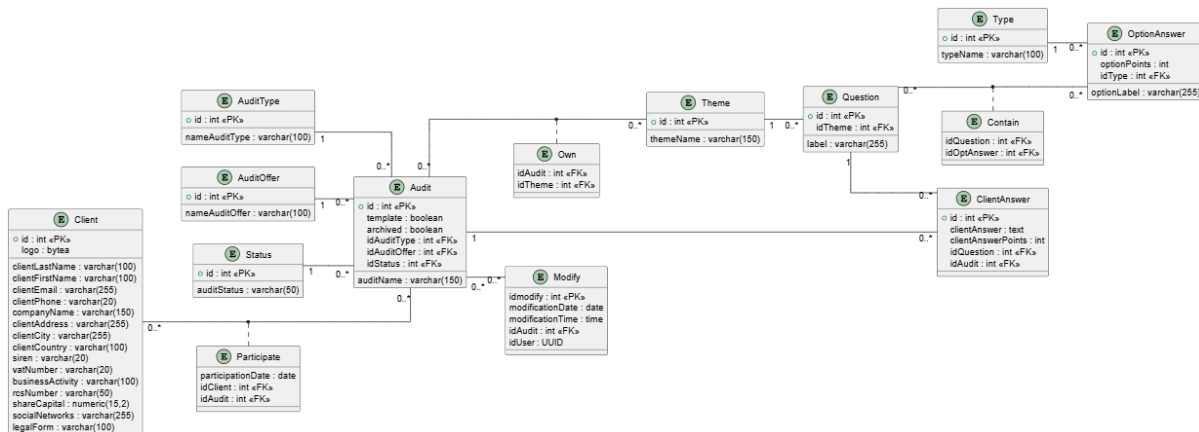
- Client
- Audit
- Utilisateur
- Template d'audit
- Thème
- Question
- Réponse client

Des tables de liaison permettent de gérer les relations plusieurs-à-plusieurs entre certaines entités (participation d'un client à un audit, association entre audits et thèmes, etc.).

1.3.3 Schéma relationnel de la base de données

Le schéma ci-dessous présente la structure relationnelle de la base de données de l'application.

Il illustre les principales entités, leurs attributs ainsi que les relations mises en place pour assurer la cohérence et l'intégrité des données.



Ce modèle permet de gérer de manière structurée les audits, les clients, les utilisateurs ainsi que les réponses associées, tout en garantissant une forte cohérence des données grâce à l'utilisation de clés étrangères et de tables de liaison.

1.3.4 Description des tables principales

Table client

La table Client contient l'ensemble des informations relatives aux clients de Bajon Consulting.

Données stockées :

- informations de contact (nom, prénom, email, téléphone)
- informations entreprise (raison sociale, SIREN, RCS, activité)
- informations légales et financières
- logo de l'entreprise

Table Audit

La table Audit représente les audits réalisés ou en cours au sein de l'application.

Elle contient :

- le nom de l'audit
- son type (AudiType)
- son offre (AudiType)
- son statut(Status)
- des indicateurs permettant de savoir si l'audit est un template ou s'il est archivé

Un audit peut :

- être associé à plusieurs clients
- contenir plusieurs thèmes
- recevoir des réponses clients

Utilisateurs (Supabase Auth)

Les utilisateurs de l'application sont gérés via le système d'authentification de Supabase (auth.users).

Les actions réalisées par les utilisateurs sur les audits sont tracées via la table Modify, qui permet de conserver un historique des modifications effectuées.

Tables Theme, Question, OptionAnswer

Les templates et audits sont structurés à l'aide de thèmes et de questions.

- Un thème regroupe plusieurs questions
- Une question peut être associée à plusieurs options de réponse
- Les types de réponses sont définis via la table Type

Cette organisation permet une structure flexible et réutilisable des audits.

Table ClientAnswer

La table ClientAnswer stocke les réponses fournies lors d'un audit.

- Chaque réponse est associée :
- à une question
- à un audit

Elle peut contenir une réponse textuelle ainsi qu'un score permettant l'analyse des résultats.

Tables de liaison (relations)

Plusieurs tables de liaison sont utilisées afin de gérer les relations complexes :

- Participate : relation entre clients et audits
- Own : association entre audits et thèmes
- Contain : association entre questions et options de réponse
- Modify : suivi des modifications effectuées par les utilisateurs sur les audits

Ces tables permettent de gérer des relations many-to-many tout en conservant une structure de données cohérente.

1.3.5 Sécurité et intégrité des données

L'intégrité des données est assurée par l'utilisation de clés primaires et étrangères.

La sécurité est renforcée grâce à :

- l'authentification Supabase
- les contraintes relationnelles PostgreSQL
- la gestion des rôles utilisateurs

Ces mécanismes garantissent la confidentialité et la fiabilité des données d'audit.

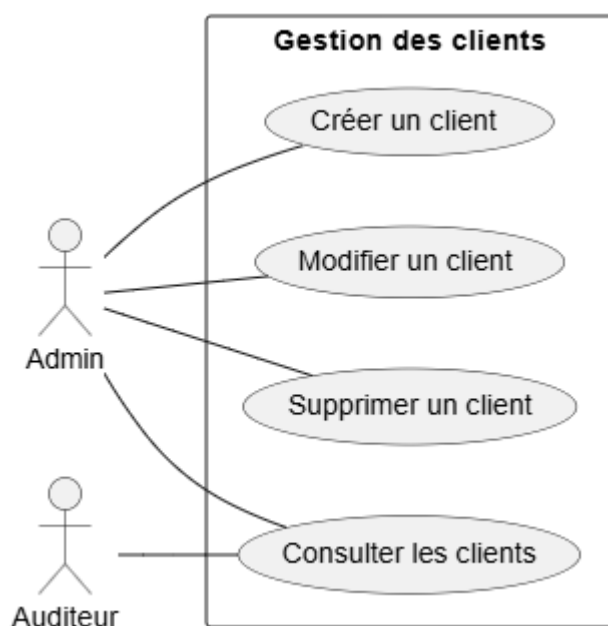
3. Use case

2.1 [US] Connexion Utilisateurs

En tant qu'auditeur je veux pouvoir me connecter avec plusieurs systèmes de connexion différents afin de pouvoir accéder au site.

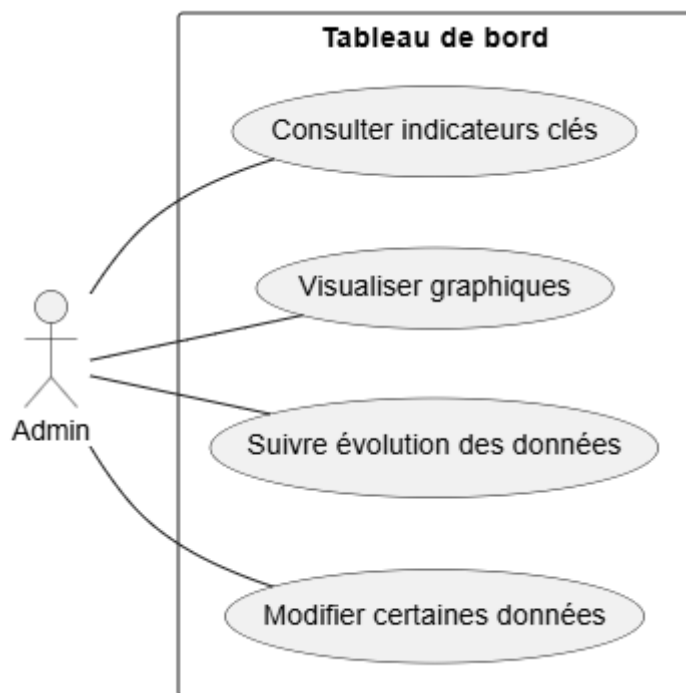
2.2 [US] CRUD Client

En tant qu'auditeur ou admin je veux pouvoir créer ,modifier ,consulter ,supprimer des clients.



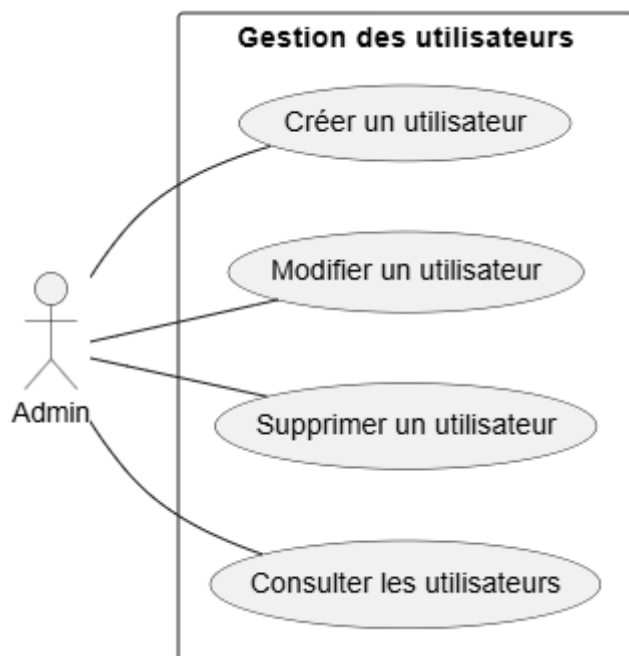
2.3 [US] Tableau de bord administrateur

En tant qu'administrateur, je veux accéder à un tableau de bord interactif afin de visualiser, suivre et analyser les données clés de l'entreprise.



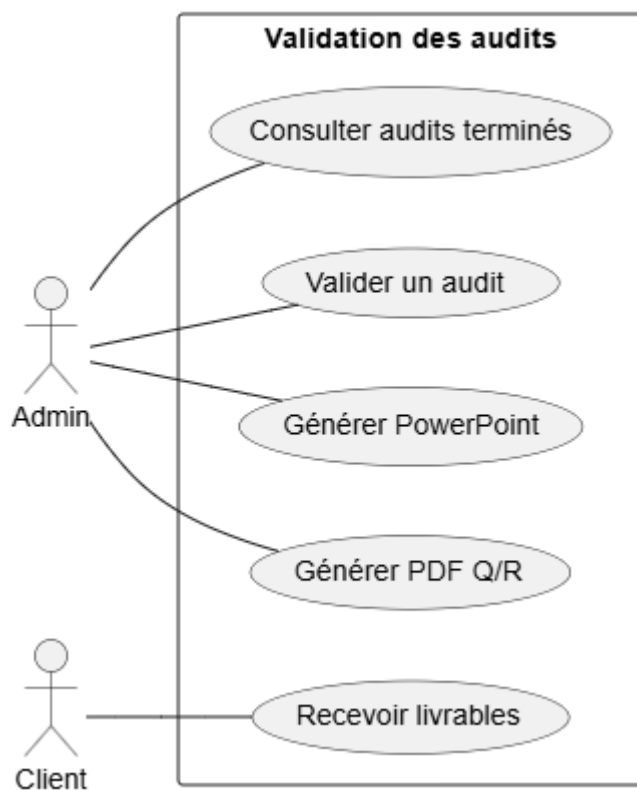
2.4 [US] CRUD Utilisateur

En tant qu'administrateur, je veux pouvoir gérer les comptes utilisateurs afin de contrôler les accès et garantir la sécurité de la plateforme.



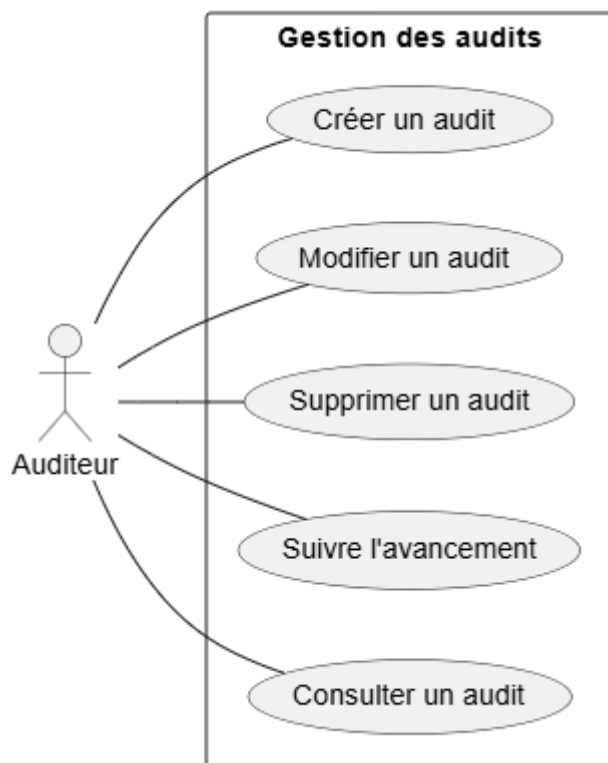
2.5 [US] Validation des audits traités

En tant qu'administrateur, je veux pouvoir valider les audits finalisés afin de livrer au client les documents de restitution (PowerPoint et PDF).



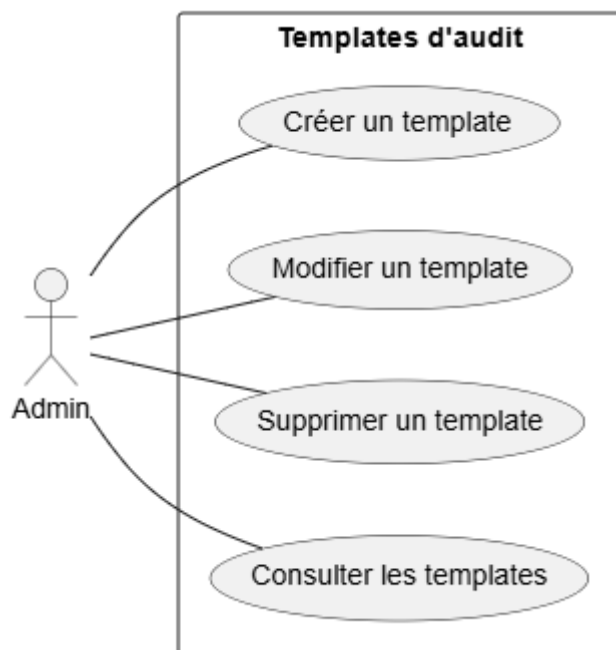
2.6 [US] Création et gestion d'audits

En tant qu'auditeur, je veux pouvoir créer, modifier et suivre les audits afin d'assurer le bon déroulement des missions d'audit.



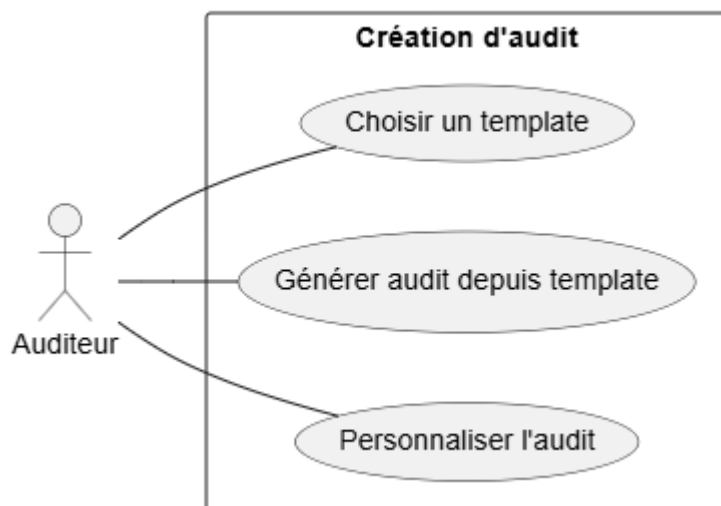
2.7 [US] Création et gestion des templates d'audit

En tant qu'administrateur, je veux pouvoir créer et modifier des templates d'audit afin de standardiser et réutiliser les structures d'audit.



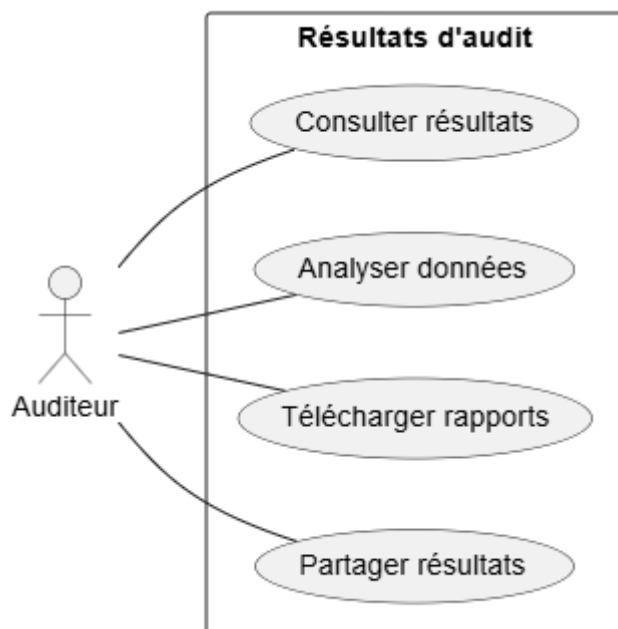
2.8 [US] Création d'audit à partir d'un template

En tant qu'auditeur, je veux pouvoir créer des audits à partir de templates afin de gagner du temps et garantir une structure cohérente.



2.9 [US] Consultation des résultats d'audit

En tant qu'auditeur, je veux pouvoir consulter et analyser les résultats des audits réalisés afin d'exploiter les données et les partager de manière sécurisée.



2.10 [US] Consultation des audits

En tant qu'auditeur, je veux pouvoir consulter la liste des audits afin de suivre leur état d'avancement et accéder rapidement à leurs détails.



4. Description des fonctionnalités

(Voir le guide utilisateur pour bien comprendre comment l'application fonctionne)

3.1. Audit

3.1.1. Créer

La création d'un audit est disponible pour tous les utilisateurs depuis la page */audits*. Lors de cette action, l'utilisateur doit remplir un formulaire contenant les informations du client ainsi que les informations nécessaires à la création de l'audit. Une fois le formulaire validé, l'utilisateur est automatiquement redirigé vers la page du formulaire d'audit afin de commencer son remplissage.

3.1.2. Modifier

La modification d'un audit est accessible à tous les utilisateurs depuis la page */audits*. Lorsqu'un audit est sélectionné pour modification, l'utilisateur est directement redirigé vers le formulaire correspondant, prérempli avec les données existantes, ce qui permet d'effectuer rapidement les changements nécessaires.

3.1.3. Consulter

La consultation d'un audit est disponible pour tous les utilisateurs. Cette fonctionnalité permet de visualiser les informations d'un audit existant sans possibilité de le modifier, garantissant ainsi l'intégrité des données lorsque l'accès est limité à la lecture seule.

3.1.4. Archiver

L'archivage d'un audit est également accessible à tous les utilisateurs. Cette action est effectuée lorsque l'audit est terminé afin de le retirer des audits actifs tout en conservant son historique pour consultation ultérieure.

3.2. Template

3.2.1. Créer

La création de templates d'audit est réservée aux administrateurs. Depuis la page */templates*, un administrateur peut créer un nouveau template en renseignant certaines informations préalables, permettant ainsi de standardiser la structure des audits futurs.

3.2.2. Modifier

La modification des templates est également réservée aux administrateurs. Cette fonctionnalité permet d'adapter ou de mettre à jour un template existant afin de répondre à de nouveaux besoins ou d'améliorer la structure des audits.

3.2.3. Consulter

La consultation des templates est accessible à tous les utilisateurs. Elle permet de visualiser le contenu d'un template sans autoriser sa modification, garantissant une utilisation cohérente des modèles définis.

3.3. Clients

3.3.1. Créer

La création d'un client s'effectue depuis la page */clients/list*. Un nouveau client peut être créé à chaque nouvel audit, bien qu'il soit également possible de sélectionner un client déjà existant afin d'éviter les doublons.

3.3.2. Modifier

La modification des informations clients est réservée aux administrateurs. Cette fonctionnalité permet de mettre à jour les données clients en cas de changement ou d'erreur.

3.3.3. Supprimer

La suppression d'un client est également réservée aux administrateurs. Elle permet de retirer un client de la base de données lorsque celui-ci n'est plus nécessaire ou a été créé par erreur.

3.4. Users

3.4.1. Créer

La gestion des utilisateurs est accessible via une page de liste dédiée, réservée aux administrateurs. Chaque utilisateur peut consulter ses informations personnelles, tandis que les administrateurs ont la possibilité de créer de nouveaux utilisateurs.

3.4.2. Modifier

La modification des utilisateurs est réservée aux administrateurs. Elle permet de mettre à jour les informations ou les droits d'un utilisateur existant.

3.4.3. Supprimer

La suppression d'un utilisateur est également une action réservée aux administrateurs. Elle permet de retirer un compte utilisateur lorsque celui-ci n'est plus nécessaire au fonctionnement de l'application.

5. Déploiement et exécution de l'application

5.1. Principe général

Le déploiement de l'application est entièrement **conteneurisé à l'aide de Docker et Docker Compose**.

Cette approche permet de garantir :

- la reproductibilité de l'environnement d'exécution,
- l'isolation des composants (frontend, backend),
- une installation simplifiée sur tout système compatible Docker,
- l'absence de dépendances locales liées aux technologies utilisées.

L'application peut ainsi être exécutée sans installer localement Node.js, Java, Maven ou toute autre dépendance métier.

5.2. Prérequis techniques

Le déploiement de l'application est conteneurisé afin de garantir la reproductibilité de l'environnement d'exécution et de simplifier l'installation.

Prérequis sur la machine hôte :

- **Docker**
- **Docker Desktop**

Aucune installation locale de Node.js / Java / Maven n'est requise pour exécuter l'application via Docker.

5.3. Architecture de déploiement

Le déploiement repose sur deux conteneurs principaux :

Frontend

- Application React + TypeScript
- Buildée en mode production via Vite
- Servie par un serveur Nginx

Backend

- API REST développée en Java avec Spring Boot
- Exposée sur le port 8080
- Responsable de la logique métier et de la génération des documents

5.4. Accès à l'application

Accès à l'application

Une fois les conteneurs démarrés :

Frontend :

- Accessible via un navigateur à l'adresse
- `http://localhost`

Backend :

- API accessible à l'adresse
- `http://localhost:8080`

Le frontend communique avec le backend via des appels HTTP internes au réseau Docker, tandis que l'authentification est réalisée via l'API Supabase.

5.5. Configuration de l'environnement

Les variables d'environnement nécessaires au fonctionnement de l'application sont définies dans un fichier `.env` utilisé lors du build et de l'exécution des conteneurs.

Exemples de variables utilisées :

- URL de l'API backend
- URL et clé publique Supabase (authentification)
- Paramètres de connexion à la base de données (backend)

Dans le cas du frontend, seules les variables préfixées par `VITE_` sont injectées lors du build.

6. Sécurité et gestion des accès

6.1. Authentification

L'authentification des utilisateurs repose sur **Supabase Auth**, utilisé via son API officielle et intégré directement au frontend de l'application.

Dans la version V1, **le mode d'authentification par email et mot de passe est le seul mécanisme implémenté**. Ce choix permet de garantir une mise en œuvre simple, fiable et adaptée à un usage interne, tout en assurant un niveau de sécurité conforme aux exigences du projet.

Supabase prend en charge automatiquement :

- la création et la gestion des comptes utilisateurs,
- la gestion des sessions et des jetons d'authentification,
- la sécurisation des échanges lors des phases de connexion et de déconnexion,
- la persistance de la session utilisateur entre les rechargements de page.

L'application consomme ces services sans stocker localement les informations sensibles, réduisant ainsi les risques liés à la gestion manuelle des identifiants.

6.2. Rôles et permissions

L'application repose sur une gestion des accès basée sur des **rôles utilisateurs**, permettant de différencier les fonctionnalités accessibles selon le profil connecté.

Deux profils sont définis dans la version V1 :

Administrateur

Le profil administrateur dispose des droits étendus nécessaires à la gestion globale de la plateforme. Il peut notamment :

- gérer les comptes utilisateurs (création, modification, suppression),
- créer, modifier et maintenir les templates d'audit,
- valider les audits finalisés,
- déclencher la génération des livrables d'audit (PowerPoint).

Auditeur

Le profil auditeur est destiné aux utilisateurs opérationnels chargés de la réalisation des audits. Il peut :

- créer et gérer des audits,
- saisir les réponses aux formulaires d'audit,
- consulter les résultats et l'état d'avancement des audits réalisés.

6.3. Protection des accès

Les fonctionnalités sensibles de l'application sont protégées à plusieurs niveaux afin de garantir la sécurité et la cohérence des données :

- **Côté frontend :**
Les routes privées sont conditionnées à l'authentification de l'utilisateur et à son rôle, empêchant l'accès aux pages non autorisées.
- **Côté backend :**
Des contrôles d'accès sont appliqués sur les endpoints critiques afin de s'assurer que seules les requêtes légitimes sont traitées.
- **Côté base de données :**
Supabase applique des règles de sécurité, des contraintes relationnelles et des politiques d'accès garantissant l'intégrité des données et limitant les accès non autorisés.

L'architecture de sécurité mise en place dans la version V1 permet d'assurer une authentification centralisée et sécurisée, une séparation claire des responsabilités entre les profils utilisateurs ainsi qu'une protection cohérente des données à chaque couche de l'application.

Cette base constitue un socle solide pour l'évolution future du projet, notamment l'ajout de nouveaux modes d'authentification ou de rôles plus fins.

Limite et évolution

Limites de la version V1 et perspectives d'évolution

La version V1 de l'application se concentre volontairement sur la mise en place des fonctionnalités essentielles. Certaines fonctionnalités avancées n'ont pas été implémentées à ce stade, notamment :

- l'authentification avancée (SSO, OAuth, authentification multi-facteurs),
- l'accès client direct aux livrables,
- l'analyse graphique avancée des résultats d'audit,
- l'export PDF automatisé des audits,
- les recommandations automatiques basées sur les scores obtenus.

Ces limitations ont été identifiées dès la phase de conception et n'affectent pas la cohérence globale de l'application. L'architecture retenue permet toutefois d'envisager l'ajout de ces fonctionnalités dans des versions ultérieures sans refonte majeure du système.

Synthèse

Ce projet a permis de concevoir et de développer une application web complète, répondant aux besoins internes de la société Bajon Consulting en matière de gestion et de suivi des audits de sécurité informatique.

La version V1 met en place une base fonctionnelle et technique solide, intégrant :

- une architecture frontend moderne en React et TypeScript,
- une API backend robuste développée avec Spring Boot,
- une base de données PostgreSQL sécurisée via Supabase,
- un système d'authentification centralisé et fiable,
- un déploiement entièrement conteneurisé grâce à Docker.

L'application permet de gérer les clients, les utilisateurs, les audits et les templates d'audit, tout en assurant une structuration claire des formulaires et des données associées. Les choix techniques réalisés garantissent une application maintenable, évolutive et adaptée à un usage professionnel interne.

Cette version constitue un socle technique fiable sur lequel pourront s'appuyer les évolutions futures de la plateforme.